DBMS – SELLER AND INVENTORY MANAGEMENT

Project Title: Etsy – Handmade & Vintage Marketplace Order Management System

Skill Task: DBMS

Week: 6

1. Database Design

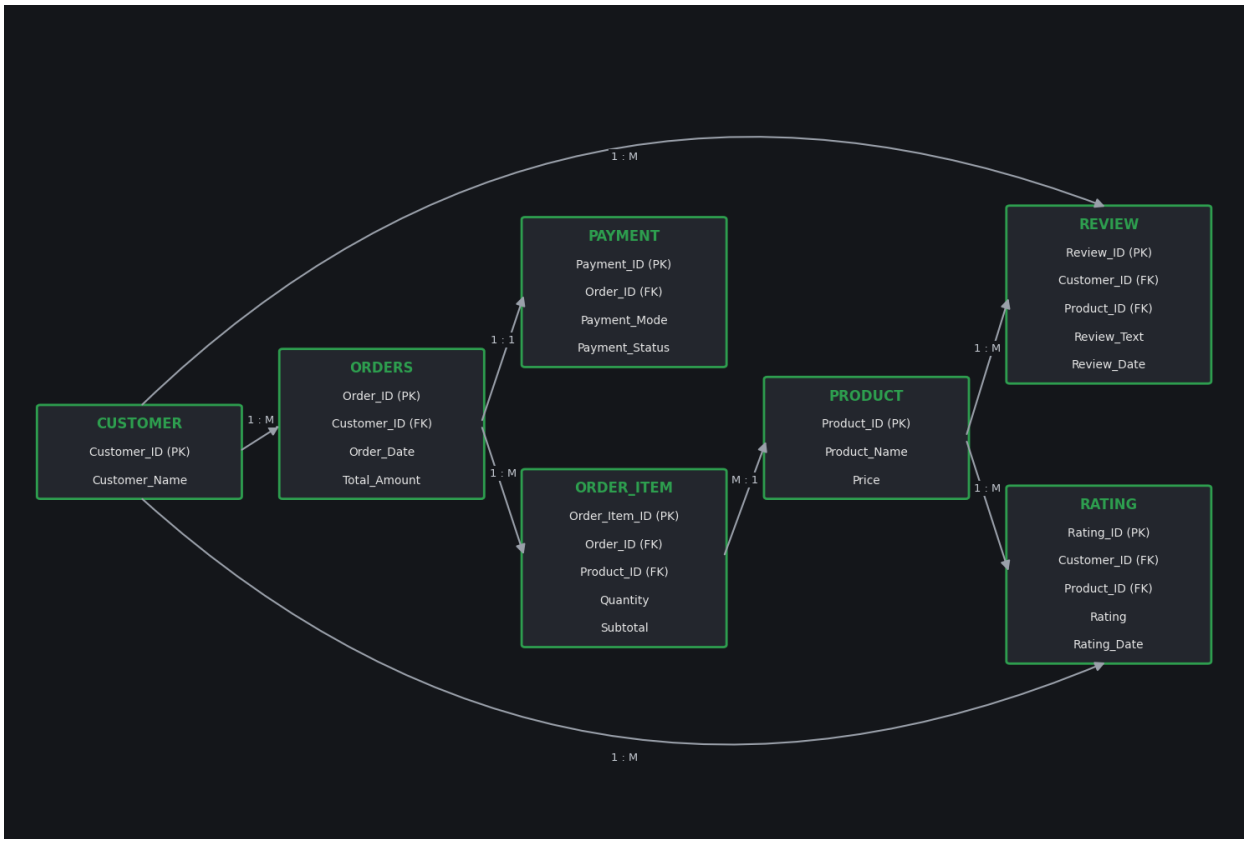
1.1 Review Table

Attribute	Data Type	Constraint
Review_ID	INT	Primary Key
Customer_ID	INT	Foreign Key
Product_ID	INT	Foreign Key
Review_Text	VARCHAR2(500)	NOT NULL
Review_Date	DATE	NOT NULL

1.2 Rating Table

Attribute	Data Type	Constraint
Rating_ID	INT	Primary Key
Customer_ID	INT	Foreign Key
Product_ID	INT	Foreign Key
Rating	INT	NOT NULL, CHECK (1 to 5)
Rating_Date	DATE	NOT NULL

2. Table Relationships



One customer can write many reviews and give many ratings. One product can receive many reviews and many ratings. Review and Rating both reference the Customer and Product tables through foreign keys.

3. Table Creation

3.1 Create Review Table

```
SQL> CREATE TABLE Review (
  Review_ID NUMBER(5) PRIMARY KEY,
  Customer_ID NUMBER(5),
  Product_ID NUMBER(5),
  Review_Text VARCHAR2(500) NOT NULL,
  Review_Date DATE NOT NULL,
  CONSTRAINT fk_review_customer FOREIGN KEY (Customer_ID)
    REFERENCES Customer(Customer_ID),
  CONSTRAINT fk_review_product FOREIGN KEY (Product_ID)
    REFERENCES Product(Product_ID)
);

Table created.
```

3.2 Create Rating Table

```
SQL> CREATE TABLE Rating (
  Rating_ID NUMBER(5) PRIMARY KEY,
  Customer_ID NUMBER(5),
  Product_ID NUMBER(5),
  Rating NUMBER(1) NOT NULL CHECK (Rating BETWEEN 1 AND 5),
  Rating_Date DATE NOT NULL,
  CONSTRAINT fk_rating_customer FOREIGN KEY (Customer_ID)
    REFERENCES Customer(Customer_ID),
  CONSTRAINT fk_rating_product FOREIGN KEY (Product_ID)
    REFERENCES Product(Product_ID)
);

Table created.
```

4. Review Data Insertion

```
SQL> INSERT INTO Review (Review_ID, Customer_ID, Product_ID, Review_Text, Review_Date)
     VALUES (601, 101, 101, 'Beautiful handmade piece, exactly as described', SYSDATE);

1 row created.

SQL> INSERT INTO Review (Review_ID, Customer_ID, Product_ID, Review_Text, Review_Date)
     VALUES (602, 102, 102, 'Lovely craftsmanship and great packaging', SYSDATE);

1 row created.

SQL> INSERT INTO Review (Review_ID, Customer_ID, Product_ID, Review_Text, Review_Date)
     VALUES (603, 103, 101, 'Very satisfied with the handmade quality', SYSDATE);

1 row created.

SQL> INSERT INTO Review (Review_ID, Customer_ID, Product_ID, Review_Text, Review_Date)
     VALUES (604, 104, 103, 'Unique design and worth the price', SYSDATE);

1 row created.
```

5. Rating Data Insertion

```
SQL> INSERT INTO Rating (Rating_ID, Customer_ID, Product_ID, Rating, Rating_Date)
     VALUES (701, 101, 101, 5, SYSDATE);

1 row created.

SQL> INSERT INTO Rating (Rating_ID, Customer_ID, Product_ID, Rating, Rating_Date)
     VALUES (702, 102, 102, 4, SYSDATE);

1 row created.

SQL> INSERT INTO Rating (Rating_ID, Customer_ID, Product_ID, Rating, Rating_Date)
     VALUES (703, 103, 101, 5, SYSDATE);

1 row created.

SQL> INSERT INTO Rating (Rating_ID, Customer_ID, Product_ID, Rating, Rating_Date)
     VALUES (704, 104, 103, 4, SYSDATE);

1 row created.
```

6. Retrieve Product Review Details

```
SQL> SET LINESIZE 200
SQL> COLUMN Customer_Name FORMAT A15
SQL> COLUMN Review_Text FORMAT A44
SQL> SELECT r.Review_ID, c.Customer_Name, r.Product_ID, r.Review_Text, r.Review_Date
     FROM Review r JOIN Customer c ON r.Customer_ID = c.Customer_ID
     ORDER BY r.Review_Date DESC;

REVIEW_ID CUSTOMER_NAME   PRODUCT_ID REVIEW_TEXT                           REVIEW_DATE
--------- --------------- ---------- ------------------------------------ -----------
      601 <name>                 101 Beautiful handmade piece, as described 06-OCT-26
      602 <name>                 102 Lovely craftsmanship, great packaging  06-OCT-26
      603 <name>                 101 Very satisfied with handmade quality   06-OCT-26
      604 <name>                 103 Unique design and worth the price      06-OCT-26
```

7. Calculate Average Product Ratings

This uses the AVG() aggregate function.

```
SQL> SELECT Product_ID, AVG(Rating) AS Average_Rating
     FROM Rating GROUP BY Product_ID;

PRODUCT_ID AVERAGE_RATING
---------- --------------
       102              4
       101              5
       103              4
```

8. Identify Highly Rated Products

Products with an average rating of 4 or above can be displayed using HAVING.

```
SQL> SELECT Product_ID, AVG(Rating) AS Average_Rating
     FROM Rating GROUP BY Product_ID
     HAVING AVG(Rating) >= 4
     ORDER BY Average_Rating DESC;

PRODUCT_ID AVERAGE_RATING
---------- --------------
       101              5
       103              4
       102              4
```

9. Product Rating Report

A combined report can be generated using the Rating and Product tables.

```
SQL> SELECT p.Product_ID, p.Product_Name,
            COUNT(r.Rating_ID) AS Total_Ratings,
            AVG(r.Rating) AS Average_Rating,
            MAX(r.Rating) AS Highest_Rating,
            MIN(r.Rating) AS Lowest_Rating
     FROM Rating r JOIN Product p ON r.Product_ID = p.Product_ID
     GROUP BY p.Product_ID, p.Product_Name
     ORDER BY Average_Rating DESC;

PRODUCT_ID PRODUCT_NAME     TOTAL_RATINGS AVERAGE_RATING HIGHEST_RATING LOWEST_RATING
---------- ---------------- ------------- -------------- -------------- -------------
       101 <product name>               2              5              5             5
       103 <product name>               1              4              4             4
       102 <product name>               1              4              4             4
```